SHRI G.S .INSTITUTE OF TECHNOLOGY AND SCIENCE
INDORE (M.P)



Department of Computer Technology and Applications
MCA First Year Semester-I
Session:- July 2025 – December 2025

Subject:- Software Engineering

Software Requirements Specification

Project:- CampusCare – College Complaint Management System

Submitted to:
Ms. Priyanka Kabra
Ms. Shweta Gupta

Submitted by:
Surbhi Lahri
Tanish Jain
Tanisha Mandhani

Table of Contents

Table of Contents **2**

1. Introduction **3**

1.1 Purpose 3

1.2 Product Scope 3

1.3 References 4

2. Overall Description **5**

2.1 Product Functions 5

3. Functional Requirements **7**

3.1 User Account Management 7

3.2 Complaint Lodging and Ticket Generation 7

3.3 Student Complaint Tracking and History 7

3.4 Admin Dashboard and Live Metrics 7

3.5 All Complaints Listing and Multi-Filtering 8

3.6 Status Transition and Resolution Management 8

3.7 Authorization and Privacy Enforcement 8

4. Non-Functional Requirements **9**

4.1 Performance Requirements 9

4.2 Reliability and Availability 9

4.3 Security Requirements 9

4.4 Usability Requirements 9

4.5 Scalability Requirements 9

4.6 Maintainability Requirements 10

4.7 Compatibility Requirements 10

1. Introduction

1.1 Purpose

This Software Requirement Specification (SRS) document specifies the requirements for the **CampusCare College Complaint Management System (Version 1.0)**. The purpose of this document is to define the complete set of software requirements for the CampusCare platform, ensuring that all stakeholders—including students, academic staff, maintenance personnel, and institutional administrators—have a clear and common understanding of its objectives, scope, and functionalities.

The scope of this SRS encompasses the full-stack software system supporting CampusCare's core operations, including:

- **Student-facing web application** for submitting, categorizing, tracking, and reviewing complaints regarding campus infrastructure, laboratories, academic facilities, and campus services.
- **Administrative management module** for reviewing, triaging, filtering, searching, and resolving complaints with verified audit trails.
- **Lifecycle state management** governing grievance transitions from initial submission to active investigation and final resolution.
- **Backend cloud services** incorporating JWT authentication, role-based authorization, automated cryptographic ticket generation, and secure data persistence using MongoDB Atlas.

1.2 Product Scope

The CampusCare College Complaint Management System is an independent, full-stack MERN software solution designed to digitize and streamline institutional grievance redressal in higher education colleges. The primary purpose of this product is to provide a fast, reliable, and completely transparent platform where students can register grievances without bureaucratic delays, while college administrators can systematically track, assign, and resolve campus issues efficiently.

Objectives and Goals

- To provide a responsive, student-centric platform where students can file complaints across distinct categories with physical location tags in under two minutes.
- To eliminate lost paperwork, informal verbal complaints, and untracked maintenance issues by generating a permanent cryptographic ticket number for every registered grievance.
- To enable real-time administrative triage, giving department heads and campus staff clear visibility over pending, in-progress, and resolved issues.

- To enforce institutional accountability by requiring mandatory resolution remarks and logging immutable audit histories with acting administrator identities and timestamps.
- To ensure strict role-based access control and horizontal data isolation, ensuring student complaint privacy while empowering administrators with campus-wide oversight.
- To provide reliable cloud-native database storage capable of scaling effortlessly across academic semesters and varying complaint volumes.

Relevant Benefits

- **Students** benefit from transparent, real-time grievance tracking, eliminating the need to physically visit departmental administrative offices.
- **Maintenance and Facilities Staff** gain structured ticket details, precise room/lab locations, and clear priority levels for rapid dispatch.
- **College Administration** obtains live metrics on recurring campus issues (such as Wi-Fi disruptions, electrical faults, or cleanliness concerns) for data-driven facility planning.
- **The Institution** benefits by strengthening internal quality assurance, fostering student satisfaction, and maintaining compliant records for accreditation audits (e.g., NAAC, NBA, and NIRF).

Relation to Institutional Strategy

The software directly supports the college's strategic roadmap of transitioning into a digitally empowered, smart academic campus. By delivering prompt issue resolution, accountability, and student-focused communication, the CampusCare application contributes to:

- Enhancing campus safety, infrastructure reliability, and hygienic standards.
- Building trust and open communication channels between students and college management.
- Supporting paperless administration and eco-friendly institutional governance.

1.3 References

1. IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
2. CampusCare Project Repository & Documentation – https://github.com/tanishjain0502/CampusCare.
3. Agile Software Development Principles – Beck, K. et al., 2001.
4. MERN Stack Architecture Reference (MongoDB, Express, React, Node.js), 2024.
5. Nielsen Norman Group – Usability Heuristics for User Interface Design, 2020.
6. OWASP Top 10 Web Application Security Risks – Open Web Application Security Project, 2021.

2. Overall Description

2.1 Product Functions

1. User Account Management and Authentication

The application enables students to register accounts using their institutional email, full name, and password. The system enforces strict role-based authorization, automatically designating public registrations as STUDENT accounts, while ADMIN privileges can only be provisioned through secure server-side initialization. Secure login is established through JSON Web Tokens (JWT) with bcrypt-encrypted passwords, ensuring credential safety.

2. Complaint Lodging and Ticket Generation

Students can log grievances across centrally defined campus categories including Classroom, Laboratory, Internet / Wi-Fi, Electrical / Maintenance, Cleanliness, Library, Hostel, Administration, and Other. Each submission incorporates priority classification (LOW, MEDIUM, HIGH), detailed descriptions (20–2000 characters), and optional physical location markers (e.g., Lab 301, Block B). Upon submission, the backend generates an immutable, collision-resistant ticket number formatted as `CC-YYYYMMDD-XXXXXX`.

3. Student Complaint Tracking and History

The student portal features a dedicated "My Complaints" repository scoped strictly to complaints authored by the logged-in student. It provides multi-criteria keyword search, status filtering, category filtering, and newest/oldest sorting. Students can inspect individual complaint pages showing complete submission details, real-time status history timelines, and administrative resolution notes upon issue closure.

4. Administrative Dashboard and Metric Calculation

College administrators have access to an executive dashboard powered by real-time MongoDB aggregations. The dashboard calculates real-time complaint counts across four key metrics: Total Complaints, Submitted / Pending, In Progress, and Resolved. Interactive metric cards serve as one-click filter shortcuts leading directly to pre-filtered complaint lists, supplemented by a live feed of the five most recent complaints.

5. All Complaints Listing and Multi-Filtering

The administrative interface provides a unified, campus-wide complaint management table. Authorized staff can execute multi-field searches across ticket numbers, complaint titles, and student names or emails simultaneously. Combined multi-parameter filters allow administrators to isolate tickets by category, priority level, or workflow status, backed by responsive server-side pagination.

6. Status Triage and Resolution Workflow

CampusCare implements a disciplined status transition workflow:

- **SUBMITTED:** Default initial state upon student creation, awaiting administrative evaluation.
- **IN_PROGRESS:** Assigned when maintenance personnel or department staff are actively investigating or rectifying the reported issue.
- **RESOLVED:** Assigned upon successful resolution. The system strictly enforces mandatory resolution remarks (minimum 5 characters) and records the resolution timestamp (`resolvedAt`).

To preserve legal and operational integrity, resolved complaints are locked against arbitrary re-modification. Every transition automatically appends an entry to the complaint's immutable status history timeline, recording the previous status, new status, acting administrator ID, and transition note.

7. Access Control, Privacy, and Data Isolation

CampusCare strictly enforces horizontal and vertical access boundaries:

- **Horizontal Isolation:** Students are restricted to viewing only their own grievances. Inquiries targeting complaints authored by another student return an HTTP 404 response to eliminate information disclosure.
- **Vertical Authorization:** All administrative endpoints (stats, all complaints, status updates) require a valid JWT containing the verified `ADMIN` role claim, rejecting student tokens with HTTP 403 Forbidden.
- **Credential Confidentiality:** Password hashes and sensitive server tokens are permanently excluded from query projections via Mongoose schema filters and serialization transforms.

8. Infrastructure and Database Reliability

The backend interfaces directly with MongoDB Atlas, maintaining persistent storage for users, complaints, and audit records. The database architecture incorporates optimized compound indexes on query fields (`ticketNumber`, `createdBy`, `status`, `priority`, and `createdAt`) to ensure sub-millisecond retrieval even as historical grievance volumes grow across academic years.

3. Functional Requirements

Functional requirements define the core operational behaviors and capabilities of the CampusCare system:

3.1 User Account Management

- The system shall allow new students to register using their full name, valid email address, and password.
- The system shall validate email syntax and reject duplicate registrations containing an existing email.
- The system shall hash passwords using bcrypt with a minimum of 10 salt rounds prior to database insertion.
- The system shall authenticate users against stored password hashes and issue signed JWT tokens on success.
- The system shall enforce role segregation, ensuring public registration endpoints exclusively provision the `STUDENT` role.

3.2 Complaint Lodging and Ticket Generation

- The system shall provide a responsive submission form validating title (5–100 characters), category, description (20–2000 characters), priority (LOW, MEDIUM, HIGH), and location (up to 100 characters).
- The system shall automatically generate a unique ticket number matching the pattern `CC-YYYYMMDD-XXXXXX` using cryptographic entropy with collision-retry protection.
- The system shall lock initial status to `SUBMITTED` and safely ignore client-supplied status or assignment payloads.
- The system shall disable submission buttons during network transit to prevent accidental duplicate ticket generation.

3.3 Student Complaint Tracking and History

- The system shall allow authenticated students to retrieve a paginated list of only their own submitted grievances.
- The system shall support real-time search across ticket numbers and complaint titles.
- The system shall provide filter dropdowns for complaint status and category, alongside sorting by newest and oldest.
- The system shall provide a dedicated detail view showing title, description, category, priority, location, created/updated timestamps, status timeline, and administrative resolution remarks.
- The system shall return HTTP 404 if a student requests a ticket belonging to another student.

3.4 Admin Dashboard and Live Metrics

- The system shall calculate and display real-time counts for Total, Submitted, In-Progress, and Resolved complaints directly from database aggregations.
- The system shall display a list of the five most recent campus grievances with live status badges.
- The system shall provide interactive stat cards that direct administrators to pre-filtered complaint lists upon clicking.

3.5 All Complaints Listing and Multi-Filtering

- The system shall allow authorized administrators to view all college complaints across all students and categories.
- The system shall provide multi-field search capabilities matching ticket numbers, complaint titles, and student names or email addresses simultaneously.
- The system shall support combined multi-parameter filtering by status (`SUBMITTED`, `IN_PROGRESS`, `RESOLVED`), category, and priority (`LOW`, `MEDIUM`, `HIGH`).
- The system shall support server-side pagination with configurable page limits (default: 10, maximum: 50) and return total record counts and page metadata.

3.6 Status Transition and Resolution Management

- The system shall allow administrators to transition complaint status between `SUBMITTED`, `IN_PROGRESS`, and `RESOLVED`.
- The system shall strictly require resolution remarks (minimum 5 characters, maximum 1000 characters) when marking a ticket as `RESOLVED`.
- The system shall store the resolution timestamp (`resolvedAt`) and resolution remarks permanently on the complaint record.
- The system shall automatically append every transition to an immutable status history array, recording previous status, new status, acting administrator ID, timestamp, and transition note.
- The system shall prevent duplicate resolution attempts or unauthorized modifications on tickets already marked `RESOLVED`.

3.7 Authorization and Privacy Enforcement

- The system shall reject unauthenticated requests to protected API routes with HTTP 401 Unauthorized.
- The system shall reject student tokens targeting administrative endpoints with HTTP 403 Forbidden.
- The system shall provision administrator credentials exclusively through server-side scripts (`npm run seed:admin`) to prevent unauthorized privilege escalation.
- The system shall permanently exclude password hashes and secret keys from all API responses via schema projection.

4. Non-Functional Requirements

Non-functional requirements describe system quality attributes, performance constraints, and operational standards:

4.1 Performance Requirements

- The system shall support at least **10,000 active students and staff members** without performance degradation.
- The application homepage and dashboard shall load within **2 seconds** under standard campus network conditions.
- API response times for complaint creation, queries, and dashboard metric calculations shall remain below **500 milliseconds**.

4.2 Reliability and Availability

- The system shall ensure **99.9% uptime** to support 24/7 student access across academic semesters.
- In case of server process interruption, cloud process managers (e.g., Render / PM2) shall automatically recover and restore service within **30 seconds**.
- Automated database backups shall occur at least **every 24 hours** via MongoDB Atlas automated snapshot management.

4.3 Security Requirements

- All network transmissions shall be encrypted over **TLS 1.3 / HTTPS** to safeguard passwords and personal grievance details.
- User passwords shall be cryptographically hashed using **bcrypt with 10–12 salt rounds**; plaintext passwords shall never be logged or stored.
- Search queries shall sanitize special regular expression characters to prevent **Regular Expression Denial of Service (ReDoS)** attacks.
- Request JSON payloads shall be limited to **10 KB** to protect against buffer overflow and resource exhaustion attacks.

4.4 Usability Requirements

- The interface shall provide a clean, modern, intuitive design adhering to institutional blue-and-white visual styling.
- All primary student tasks (filing a complaint, tracking status) shall be achievable within **2 clicks from the dashboard**.
- The system shall provide clear visual cues including status color badges, priority indicators, loading spinners, and actionable error alerts.

4.5 Scalability Requirements

- The backend architecture shall remain completely stateless, enabling horizontal scaling across multiple container instances.
- Database indexes on ticket numbers, creator IDs, statuses, and timestamps shall maintain logarithmic query performance as records expand.

4.6 Maintainability Requirements

- The codebase shall follow clean modular MERN architecture with distinct separation of models, controllers, routes, middleware, and frontend components.
- Complaint categories, priorities, and status values shall be centrally defined in `complaintConstants.js` to allow effortless institutional customization.
- Comprehensive automated test suites (student complaint tests, admin tests, and end-to-end security suites) shall be maintained to guarantee regression-free future updates.

4.7 Compatibility Requirements

- The web application shall be fully compatible with major modern browsers: **Google Chrome (v90+)**, **Mozilla Firefox (v88+)**, **Apple Safari (v14+)**, and **Microsoft Edge (v90+)**.
- The responsive frontend layout shall flawlessly support all standard viewport sizes across **Android (v9.0+)**, **iOS (v13+)**, Windows, macOS, and Linux devices.